

# App Dev 2 Project Report

## 1. Student Details

**Name:** Kirti Gupta

**Roll Number:** 21f1000474

**Email:** 21f1000474@ds.study.iitm.ac.in

**About Me:** I am a Data Science student at IIT Madras with a strong interest in AI / ML, and Data Analytics. I enjoy working on real-world projects that turn data into meaningful insights and intelligent solutions.

---

## 2. Project Details

**Project Title:** Hospital Management System V2

### Problem Statement:

Hospitals need efficient systems to manage patients, doctors, hospital appointments, and treatments. Currently, many hospitals use manual registers or disconnected software, which makes it difficult to manage records, avoid scheduling conflicts, and track patient history.

### Approach:

This project is implemented as a role-based Hospital Management System using a Flask REST API backend and a Vue.js frontend. The system supports three roles: Admin, Doctor, and Patient. The backend uses SQLAlchemy models with programmatic database creation, JWT-based authentication, Redis caching, and Celery + Redis for asynchronous and scheduled background jobs. The frontend is built using Vue.js, Vue Router, Axios, and Bootstrap 5 for a responsive user interface.

The project focuses on:

- secure role-based access
  - doctor availability management
  - appointment booking/rescheduling/cancellation
  - treatment record management
  - patient history access
  - CSV export
  - scheduled reminders and monthly reports
- 

## 3. AI/LLM Declaration

I used **ChatGPT (GPT-5)** as a development assistant during the implementation of the **MAD-2 Hospital Management System**. AI support was mainly used for resolving errors, improving structure, and assisting with some framework integrations. The approximate areas where AI assistance was used include:

- **Flask API Structure (≈10–12%)**  
Guidance on organizing REST API routes, structuring modular Flask blueprints, and resolving issues related to request handling and role-based access control.
- **Vue.js + Axios API Communication (≈20–25%)**  
Most of the Vue frontend structure, component patterns, and API communication logic were implemented with assistance from LLM suggestions.
- **Celery + Redis Background Jobs (≈10–12%)**  
LLM assistance was used to configure Celery workers, Redis integration, and background tasks such as reminders, monthly reports, and CSV export jobs.
- **Testing / Debugging (≈5%)**  
Help interpreting error tracebacks, diagnosing cache issues, resolving token authorization problems, and identifying logical inconsistencies between frontend and backend APIs.

Overall AI/LLM usage in this project is **approximately 45–50%**. The majority of AI assistance was used for **Vue frontend development, API communication patterns, and configuring Celery–Redis background job infrastructure**. AI was primarily used for guidance, debugging, and improving code structure.

**All core implementation logic, database design, appointment workflows, booking/rescheduling logic, role-based routing, and end-to-end integration were implemented manually.**

AI was *not* used for automatic code generation of business logic.

---

## 4. Technologies and Frameworks Used

| Technology / Library | Purpose                                            |
|----------------------|----------------------------------------------------|
| Flask                | Backend framework for building REST APIs           |
| SQLAlchemy           | ORM for database models and queries                |
| SQLite               | Lightweight database for application data          |
| Vue.js               | Frontend framework for building the user interface |
| Vue Router           | Handles client-side page routing                   |
| Axios                | Sends API requests from frontend to backend        |
| Bootstrap 5          | UI styling and responsive layout                   |
| Flask-JWT-Extended   | Token-based authentication                         |
| Celery               | Executes background tasks                          |
| Redis                | Message broker for Celery tasks                    |
| Flask-Caching        | Caches frequently used API responses               |

---

## 5. Database Schema / ER Diagram

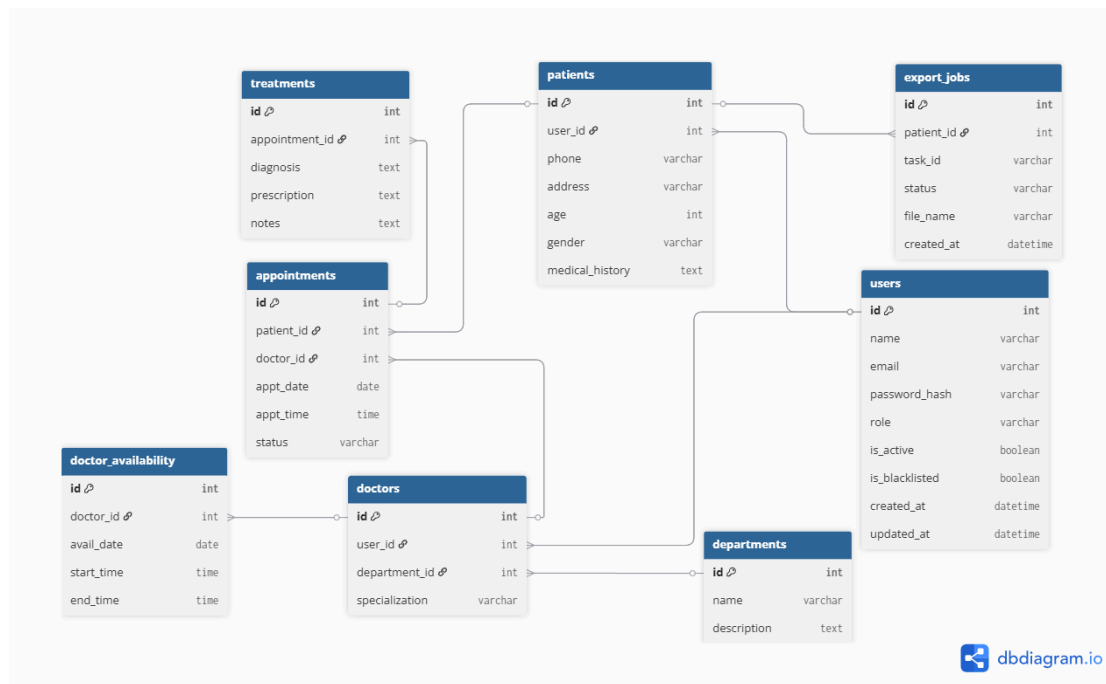
### Tables:

1. **User** — stores login and authentication details (id, name, email, password\_hash, role, is\_active, is\_blacklisted)
2. **Doctor** — stores doctor-specific data (id, user\_id, specialization, department\_id)
3. **Patient** — stores patient-specific data (id, user\_id, phone, address, age, gender, medical\_history)
4. **Department** — stores hospital departments (id, name, description)
5. **Availability** — stores doctor availability slots (id, doctor\_id, avail\_date, start\_time, end\_time)
6. **Appointment** — stores appointment records (id, patient\_id, doctor\_id, appt\_date, appt\_time, status)
7. **Treatment** — stores diagnosis and prescription details (id, appointment\_id, diagnosis, prescription, notes)
8. **ExportJob** — stores patient CSV export job details (id, patient\_id, task\_id, status, file\_name, created\_at, completed\_at, error\_message, started\_at)

### Relationships:

- **One-to-One:** User -> Doctor
- **One-to-One:** User -> Patient
- **One-to-Many:** Department -> Doctor
- **One-to-Many:** Doctor -> DoctorAvailability
- **One-to-Many:** Doctor -> Appointment
- **One-to-Many:** Patient -> Appointment
- **One-to-One:** Appointment -> Treatment
- **One-to-Many:** Patient -> ExportJob

Note: Blacklist and active/inactive status are maintained at the **User** level, not separately inside Doctor or Patient tables.



( ER diagram from [dbdiagram.io](https://dbdiagram.io) )

---

## 6. API Resource Endpoints

| Endpoint                                                         | Method      | Role(s) Allowed        | Description                            |
|------------------------------------------------------------------|-------------|------------------------|----------------------------------------|
| <code>/api/auth/register</code>                                  | POST        | patient                | Register a new patient                 |
| <code>/api/auth/login</code>                                     | POST        | admin, doctor, patient | Authenticate user and return JWT token |
| <code>/api/auth/me</code>                                        | GET         | authenticated users    | Return current logged-in user details  |
| <code>/api/admin/summary</code>                                  | GET         | admin                  | Admin dashboard summary                |
| <code>/api/admin/departments</code>                              | GET, POST   | admin                  | List or create departments             |
| <code>/api/admin/doctors</code>                                  | GET, POST   | admin                  | List/search doctors or create doctor   |
| <code>/api/admin/doctors/{doctor_id}</code>                      | PUT, DELETE | admin                  | Update or delete doctor                |
| <code>/api/admin/doctors/{doctor_id}/blacklist</code>            | PATCH       | admin                  | Blacklist/unblacklist doctor           |
| <code>/api/admin/patients</code>                                 | GET         | admin                  | List/search patients                   |
| <code>/api/admin/patients/{patient_id}/blacklist</code>          | PATCH       | admin                  | Blacklist/unblacklist patient          |
| <code>/api/admin/appointments</code>                             | GET         | admin                  | View appointments                      |
| <code>/api/admin/appointments/{appointment_id}/cancel</code>     | PATCH       | admin                  | Cancel appointment                     |
| <code>/api/admin/appointments/{appointment_id}/complete</code>   | PATCH       | admin                  | Mark appointment completed             |
| <code>/api/doctor/dashboard</code>                               | GET         | doctor                 | Doctor dashboard                       |
| <code>/api/doctor/appointments</code>                            | GET         | doctor                 | View assigned appointments             |
| <code>/api/doctor/appointments/{appointment_id}/status</code>    | PATCH       | doctor                 | Update appointment status              |
| <code>/api/doctor/appointments/{appointment_id}/treatment</code> | GET, PUT    | doctor                 | View/add/update treatment              |

|                                                |           |         |                                   |
|------------------------------------------------|-----------|---------|-----------------------------------|
| /api/doctor/patients                           | GET       | doctor  | View assigned patients            |
| /api/doctor/patients/{patient_id}/history      | GET       | doctor  | View patient treatment history    |
| /api/doctor/availability                       | GET, POST | doctor  | View/add availability slots       |
| /api/doctor/availability/{slot_id}             | DELETE    | doctor  | Delete availability slot          |
| /api/doctor/profile                            | GET, PUT  | doctor  | View/update doctor profile        |
| /api/patient/dashboard                         | GET       | patient | Patient dashboard                 |
| /api/patient/profile                           | GET, PUT  | patient | View/update patient profile       |
| /api/patient/doctors                           | GET       | patient | Search doctors by name/department |
| /api/patient/doctors/{doctor_id}               | GET       | patient | View doctor details and slots     |
| /api/patient/appointments                      | GET, POST | patient | View or book appointments         |
| /api/patient/appointments/{appt_id}            | GET       | patient | View a specific appointment       |
| /api/patient/appointments/{appt_id}/reschedule | PUT       | patient | Reschedule appointment            |
| /api/patient/appointments/{appt_id}/cancel     | PUT       | patient | Cancel appointment                |
| /api/patient/history                           | GET       | patient | View completed treatment history  |
| /api/patient/exports/treatments                | POST      | patient | Trigger CSV export                |
| /api/patient/exports                           | GET       | patient | View export jobs                  |
| /api/patient/exports/{job_id}/download         | GET       | patient | Download completed CSV export     |

---

## 7. Architecture and Features (optional)

### Architecture Overview:

- **backend/app.py** - Flask app entry point and configuration
- **backend/models.py** - database models and relationships
- **backend/routes.py** - all role-based API routes
- **backend/tasks.py** - Celery background tasks
- **backend/notifier.py** - notification helpers
- **frontend/src/** - Vue components, router, services, and views
- **Redis** - caching and Celery broker
- **SQLite** - local application database

### Implemented Features:

- JWT-based login for Admin, Doctor, and Patient
  - Patient self-registration
  - Programmatic database creation
  - Pre-seeded admin user
  - Admin dashboard with hospital summary
  - Department management
  - Doctor management
  - Patient search and monitoring
  - Blacklisting / activation controls
  - Doctor availability management for upcoming dates
  - Appointment booking, cancellation, and rescheduling
  - Conflict prevention for overlapping appointments
  - Doctor treatment entry and consultation notes
  - Patient treatment history view
  - Doctor access to patient history
  - CSV export of patient treatment records
  - Scheduled background jobs:
    - daily reminder job
    - monthly doctor activity report
    - Redis-based caching for selected API responses
  - Vue.js + Bootstrap responsive frontend
- 

## 8. Video Presentation

### Demo Link:

[https://drive.google.com/file/d/1ldP\\_IDp60zuP9Gb9Mi7atCGLgDeiDtPZ/view?usp=sharing](https://drive.google.com/file/d/1ldP_IDp60zuP9Gb9Mi7atCGLgDeiDtPZ/view?usp=sharing)

(Accessible to within IITM organization.)